MID-SEMESTER EXAMINATION, April-2025
Compilers:Principles,Techniques, and Tool (CSE 3739)

Program: B.Tech (CSE, CSIT) **Semester:6th**
Full Marks: 30 **Time: 2 Hours**

Subject/Course Learning Outcome	*Taxonomy Level	Ques. Nos.	Marks
Understand the overview of programming languages, language processors and the structure of a compiler.	L2,L3 L1,L2 L2,L3	1.a 1.b 1.c	6
Acquire knowledge in theory of computation and their role in designing different types of tokens generated by lexical analyzer.	L3 L3,L4 L1,L2	2.a 2.b 2.c	6
Understand the role of Parser(s) (LL, SLR, CLR and LALR) and its types i.e. Top-down and Bottom-up parsers.	L3,L4 L4,L5 L3,L4 L3,L4 L4,L5 L4,L5	3.a 3.b 3.c 4.a 4.b,c 5.a,b, c	18

*Bloom's taxonomy levels: Remembering (L1), Understanding (L2), Application (L3), Analysis (L4), Evaluation (L5), Creation (L6)

Answer all questions. Each question carries equal mark.

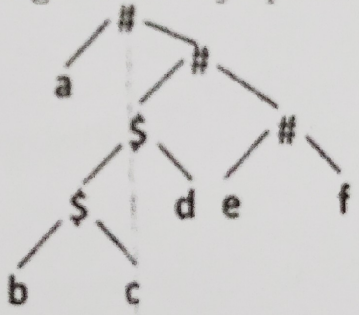
1	(a)	Consider the following fragment of 'C' code: **float a,b;** **a = a * - 50 + b + 3;** Write the output **of 1st Four phases of the compiler** for the above arithmetic expression in the 'C' code.	2
	(b)	Write short notes on: i) Symbol table ii) Input buffering	2
	(c)	Explain the language processing system for the modern computer programming languages.	2

2.	(a)	Write the regular expression for the **floating point number with exponent**. The example of some floating point numbers with exponent are some strings such as 6.5897E4, 1.8569E-4, 2.54e15 or 3.54e-15. [**Hint:** At least one digit should be before and after the dot (.)]	2
	(b)	Construct a transition diagram (finite state automaton) that represents the lexical recognition of a **floating- point number with an exponent.**	2
	(c)	Find the number of tokens in the given C statement. **printf("pt = %d, &pt = %x", pt, &pt);** **[GATE 2000]**	2
3.	(a)	Consider two grammars G with the production rules given below: **G:**S→ if E then S | if E then S else S | a E→ b where **if, then, else, a,b,c** are the terminals. Determine whether the given grammar G is ambiguous. **[GATE 2025]**	2
	(b)	Determine whether the given grammar G in Question 3(a) is LL(1) grammar. [GATE 2025]	2
	(c)	Consider the following context-free grammar where the start symbol is S and the set of terminals is {a,b,c,d}. **S→AaAb | BbBa** **A→cS | ε** **B→dS | ε** The following is a partially-filled LL(1) parsing table.	2

	a	b	c	d	$
S	S → AaAb	S → BbBa	**(1)**	**(2)**	
A	A →ε	**(3)**	A → cS		
B	**(4)**	B →ε		B → dS	

Write down the CORRECT productions for the numbered cells in the parsing table? **[GATE 2024]**

4.	(a)	Compute the FIRST( ) and FOLLOW( ) of the given grammar: **S→Aba | bCA** **A→cBCD | ε** **B→CdA | ad**	2

		C→eC | ε **D→bsf | a**	
	(b)	The attributes of three arithmetic operators in some programming language are given below. **perator** / **Precedence** / **Associativity** / **Arity** **+** / **High** / **Left** / **Binary** **_** / **Medium** / **Right** / **Binary** ***** / **Low** / **Left** / **Binary** Compute the value of the expression **2 – 5 + 1 – 7 * 3** and create the parse tree for this expression. **[GATE 2016]**	2
	(c)	Consider the following parse tree for the expression **a#bcd#e#f**, involving two binary operators **$** and **#**.  Determine which operator has the high precedence and what is the associativity of the two operators $ and #. **[GATE 2018]**	2
5.	(a)	Consider the following grammar with production rules as : E → T + E | T T →id Compute the canonical collection of LR(0) items of the grammar.	2
	(b)	Justify that the grammar given in question 5(a) is NOT LR (0) grammar.	2
	(c)	Construct the SLR(1) parsing table for the grammar given in question 5(a).	2
		End of Questions	

END-SEMESTER EXAMINATION, May-2025
Compilers:Principles,Techniques, and Tool (CSE 3739)

Programme: B.Tech(CSE,CSIT) **Semester: 6th**
Full Marks: 60 **Time: 3 Hours**

Subject/Course Learning Outcome	*Taxonomy Level	Ques. Nos.	Marks
Understand the overview of programming languages, language processors and the structure of a compiler.	L3, L4	1,2	12
Acquire knowledge in theory of computation and their role in designing different types of tokens generated by lexical analyzer.	L3	3,4	12
Understand the role of Parser(s) (LL, SLR, CLR and LALR) and its types i.e. Top-down and Bottom-up parsers.	L6	5,6,7, 8	24
Apply and evaluate syntax directed translation schemes, synthesized attributes, inherited attributes, and different techniques for symbol table organization	L5	9	6
Analyze the generation of various intermediate codes and the process of their optimization.	L4	10. a,c	4
Understand the target machine's run time environment, and its instruction set for code generation and techniques used for code optimization	L3	10. b	2

*Bloom's taxonomy levels: Remembering (L1), Understanding (L2), Application (L3), Analysis (L4), Evaluation (L5), Creation (L6)

Answer all questions. Each question carries equal mark.

1.	(a)	Write the output of 1st four phases of the compiler for the following arithmetic expression **x = (a+b)* -c/d**	2
	(b)	Consider the following fragment of 'C' code: **float a,b;** **a = a * - 50 + b + 3;** Write the output of **last 3-phases of the compiler** for the above arithmetic expression in the 'C' code.	2
	(c)	Describe the importance of symbol table during compilation process of a program.	2

2.	(a)	Provide a comparative analysis of a compiler and an interpreter, highlighting their differences and similarities.	2
	(b)	Define a hybrid compiler and explain its working with the help of a diagram.	2
	(c)	Consider the following C program: **#define a (x+1)** **int x = 2;** **void b( ) { x = a; printf (" %d\n" , x) ; }** **void c( ) { int x = 1; printf (" %d\n "), a; }** **void main( ) { b( ) ; c( ) ; }** Identify the output of the program and explain how the macro #define a (x+1), affects the scope of the variables **a** and **x** in the functions **b()** and **c()**.	2
3.	(a)	Define Token, Lexeme and Pattern with suitable examples.	2
	(b)	Write regular definitions for the valid lexical structure of floating-point constants in the C programming language.	2
	(c)	The pattern for the token relop is defined as: ***relop*** → **< | > | < = | >= | = | < >** Construct the state transition diagram that will recognize the token ***relop.***	2
4.	(a)	What are lexical errors in the context of lexical analysis? Describe how panic mode recovery is used to handle such errors.	2
	(b)	Write regular definitions for the valid lexical structure of identifiers in the C programming language.	2
	(c)	Consider the following C program, identify the tokens, and count the total number of tokens. **int main(i, j)** **{** **int i, j;** **return i>j? i : j;** **}**	2
5.	(a)	Consider the language **L** defined by the regular expression **(a|b)*a (a|b)** over the alphabet {a,b}. Construct an NFA that accepts this language.	2
	(b)	Construct the equivalent **DFA** from the **NFA** constructed in Question 5(a).	2
	(c)	Write the regular definition for **natural numbers** that is divisible by 5. A number is considered divisible by 5 if its last digit is either 0 or 5. Examples of such numbers include 5280, 115, 105, 5, and 2025	2
6.	(a)	Check the given CFG is having common prefix or not. If yes, remove it from the grammar and rewrite the grammar. **E→ iEtS| iEtMeS|c** **M→ b**	2

	(b)	Consider the Context-free Grammar: **S → SS+ | SS* | a** Give a rightmost derivation and construct the parse tree for the string ***w = aa + a****	
	(c)	Let G be the following context-free grammar, where E is the start symbol of G ; ***E → E # E | E $ E | E ? E | E @ E | id*** Construct an unambiguous grammar G' for above ambiguous grammar G with reference to the operator table given below. [**Hint**: Assume **#, $, ? and @** are binary operators]	2

operator	Associativity	precedence
@	right to left	lowest ↓ highest
#	right to left	
?	left to right	
$	left to right	

7.	(a)	Given the context-free grammar with its productions: ***S → S (S) S | ε*** Show that it is not suitable for a top-down predictive parser. Identify the problem and correct it by rewriting the grammar.	2
	(b)	Consider the following **context-free grammar**: **S→AA** **A→ aA|b** Construct the **canonical collection of LR(0) items** for this grammar and draw the **LR(0) transition diagram (DFA of items)**.	2
	(c)	Construct the **LR(0) parsing table** for the grammar given in Question 7(b) and demonstrate the **parsing actions** (shift, reduce, accept, error) for the input string: **aabb**	2
8.	(a)	Define **handle pruning** in the context of bottom-up parsing. Consider the following grammar: **E → E + n | E × n | n** Determine the **handles** and Construct the **bottom-up (shift-reduce) parse** for the string: **n + n × n**	2
	(b)	Consider the following context-free grammar, **E→ E - T | T** **T → T * F | F** **F → a** Find the canonical collection of LR(1) sets of items and construct the transition diagram.	2
	(c)	Construct the CLR(1) parsing table for the Grammar defined in Question 8(b).	2
9.	(a)	Consider the following grammar with syntax-directed translation rules: **1) S → S # T { S.val = S.val * T.val }** **2) S → T { S.val = T.val }** **3) T → T % R { T.val = T.val ÷ R.val }** **4) T → R { T.val = R.val }** **5) R → id { R.val = id.val }**	2

		Here # and % are operators and id is a token that represents an integer and id.val represents the corresponding integer value. Draw the **annotated parse tree** for the input expression: **20#10%5#8%2%2** and compute the **final value** of **S.val.**	
	(b)	Given the following syntax-directed translation rules: **Rule 1: R → AB { B.i = R.i − 1; A.i = B.i; R.i = A.i + 1; }** **Rule 2: P → CD { P.i = C.i + D.i; D.i = C.i + 2; }** **Rule 3: Q → EF { Q.i = E.i + F.i; }** Assume all attributes are integers. For each rule, identify which attributes are inherited and which are synthesized.	2

	(c)	Consider the following intermediate code segment:
		x = u - t;
		y = x * v;
		x = y + w;
		y = t - z;
		y = x * y;
		Construct the **Directed Acyclic Graph (DAG)** representation for the above code segment.
			2

10	(a)	L1: t1 = -1 L2: t2 = 0 L3: t3 = 0 L4: t4 = 4 * t3 L5: t5 = 4 * t2 L6: t6 = t5 * M L7: t7 = t4 + t6 L8: t8 = a[t7] L9: if t8 <= max goto L11 L10: t1 = t8 L11: t3 = t3 + 1 L12: if t3 < M goto L4 L13: t2 = t2 + 1 L14: if t2 < N goto L3 L15: max = t1 Divide the given pseudo-code into basic blocks. Then, determine the number of instructions in the largest basic block.	2
	(b)	Generate the assembly language code for the given intermediate code: t1 = a+b t2 = c+d t3 = e-t2 t4 = t1-t3	2
	(c)	Discuss various Loop optimization techniques with example.	2
		End of Questions	